

Perfecto. Iniciamos con **Paso 23B: convertir los ~98.1M registros account-month en una tabla party-month.**

La fuente validada es:

bu_frd.model_trades_transition_13m_2022_2025

y construiremos:

bu_frd.model_party_behavior_monthly_2022_2025

La granularidad final será:

party_id + observation_date + feature_month

Cada persona debe terminar con **12 meses analíticos**: 9 PRIOR_9M + 3 RECENT_3M. El mes SEED ya cumplió su función para calcular los LAG() y desde aquí lo excluimos.

23B — Crear party-month

DROP TABLE IF EXISTS

bu_frd.model_party_behavior_monthly_2022_2025;

CREATE TABLE

bu_frd.model_party_behavior_monthly_2022_2025

USING PARQUET

AS

SELECT

```
/* =====  
MODEL / PARTY IDENTIFIERS  
===== */
```

```
party_id,  
observation_date,  
observation_year,  
UPPER(model_split) AS model_split,  
label_3m,  
cohort_role,
```

```
/* =====  
MONTH / WINDOW  
===== */
```

```
feature_month,  
month_offset,  
feature_window,
```

```
/* =====  
ACCOUNT EXPOSURE  
===== */
```

```
COUNT(DISTINCT acct_id)  
AS reported_accounts,
```

```
COUNT(  
DISTINCT CASE  
WHEN active_account_flag = 1  
THEN acct_id  
END  
) AS active_accounts,
```

```
/* =====  
MOP TRANSITIONS  
===== */
```

```
SUM(dpd_deterioration_flag)  
AS deterioration_count,
```

```
SUM(dpd_improvement_flag)  
AS improvement_count,
```

```
SUM(entered_active_adverse_flag)  
AS adverse_entry_count,
```

```
SUM(adverse_resolved_flag)  
AS adverse_resolved_count,
```

```
SUM(became_unrated_flag)  
AS became_unrated_count,
```

```
SUM(left_unrated_flag)  
AS left_unrated_count,
```

```
/* Maximum positive DPD movement in month */
```

```
MAX(  
CASE  
WHEN dpd_level_change > 0  
THEN dpd_level_change  
ELSE 0  
END  
) AS max_dpd_jump,
```

```

/* =====
BALANCE
Only active accounts
===== */

```

```

AVG(
  CASE
    WHEN active_account_flag = 1
    THEN curr_bal_amt
  END
) AS avg_balance_active,

```

```

SUM(
  CASE
    WHEN active_account_flag = 1
    THEN curr_bal_amt
  END
) AS total_balance_active,

```

```

/* =====
PAST DUE
===== */

```

```

AVG(
  CASE
    WHEN active_account_flag = 1
    THEN past_due_amt
  END
) AS avg_past_due_active,

```

```

SUM(
  CASE
    WHEN active_account_flag = 1
    THEN past_due_amt
  END
) AS total_past_due_active,

```

```

/* =====
DELINQUENCY SHARE

```

Denominator:
 active accounts with known ordinal
 delinquency level 01-06.

07 / 08 / UR / adverse MOP codes are NOT
automatically treated as non-delinquent.

===== */

```
COUNT(  
  DISTINCT CASE  
    WHEN active_account_flag = 1  
      AND current_delinquency_level IS NOT NULL  
    THEN acct_id  
  END  
) AS rated_active_accounts,
```

```
COUNT(  
  DISTINCT CASE  
    WHEN active_account_flag = 1  
      AND current_delinquency_level > 0  
    THEN acct_id  
  END  
) AS delinquent_active_accounts,
```

```
CASE  
  WHEN COUNT(  
    DISTINCT CASE  
      WHEN active_account_flag = 1  
        AND current_delinquency_level IS NOT NULL  
      THEN acct_id  
    END  
  ) > 0  
  THEN  
    1.0 *  
    COUNT(  
      DISTINCT CASE  
        WHEN active_account_flag = 1  
          AND current_delinquency_level > 0  
        THEN acct_id  
      END  
    )  
    /  
    COUNT(  
      DISTINCT CASE  
        WHEN active_account_flag = 1  
          AND current_delinquency_level IS NOT NULL  
        THEN acct_id  
      END  
    )  
  ELSE NULL  
END AS delinquent_account_share,
```

```

/* =====
UTILIZATION

Only active revolving / line-of-credit
accounts with credit_limit > 0.
===== */

COUNT(
  DISTINCT CASE
    WHEN active_account_flag = 1
    AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
      IN ('C','R')
    AND cr_lmt_amt > 0
    THEN acct_id
  END
) AS utilization_eligible_accounts,

AVG(
  CASE
    WHEN active_account_flag = 1
    AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
      IN ('C','R')
    AND cr_lmt_amt > 0
    THEN
      1.0 * curr_bal_amt / cr_lmt_amt
    END
) AS avg_utilization

FROM
  bu_frd.model_trades_transition_13m_2022_2025

WHERE
  feature_window IN ('PRIOR_9M','RECENT_3M')

GROUP BY
  party_id,
  observation_date,
  observation_year,
  UPPER(model_split),
  label_3m,
  cohort_role,
  feature_month,
  month_offset,
  feature_window;

```

Aquí **no hacemos todavía ninguna imputación ni threshold**. Si utilization no existe para una persona porque no tiene cuentas **C/R** elegibles, queda **NULL**, no **0**.

QA 1 — tamaño

```
SELECT
  COUNT(*) AS party_month_rows,
  COUNT(DISTINCT party_id) AS parties,
  MIN(feature_month) AS min_feature_month,
  MAX(feature_month) AS max_feature_month

FROM
  bu_frd.model_party_behavior_monthly_2022_2025;
```

Esperamos:

party_month_rows = 8,263,068
parties = 688,589

porque:

$688,589 \times 12 = 8,263,068$

QA 2 — ventanas

```
SELECT
  feature_window,
  COUNT(*) AS rows,
  COUNT(DISTINCT party_id) AS parties

FROM
  bu_frd.model_party_behavior_monthly_2022_2025
```

GROUP BY feature_window

ORDER BY feature_window;

Esperamos:

PRIOR_9M
rows = 6,197,301
parties = 688,589

RECENT_3M
rows = 2,065,767
parties = 688,589

Porque:

$688,589 \times 9 = 6,197,301$

y:

688,589×3=2,065,767688,589\times3=2,065,767

QA 3 — exactamente 12 meses por party

WITH x AS (

```
SELECT
  party_id,
  observation_date,
  COUNT(*) AS months
```

```
FROM
  bu_frd.model_party_behavior_monthly_2022_2025
```

```
GROUP BY
  party_id,
  observation_date
```

)

```
SELECT
  MIN(months) AS min_months,
```

```
  percentile_approx(
    months,
    0.50
  ) AS median_months,
```

```
  MAX(months) AS max_months,
```

```
  SUM(
    CASE
      WHEN months <> 12
      THEN 1
      ELSE 0
    END
  ) AS parties_not_12_months
```

FROM x;

Esperamos:

min_months	= 12
median_months	= 12
max_months	= 12
parties_not_12_months	= 0

QA 4 — cobertura de variables

Este resultado es muy importante antes de calcular Recent 3m - Prior 9m.

```
SELECT
  model_split,
  label_3m,

  COUNT(*) AS party_months,

  /* Balance */

  SUM(
    CASE
      WHEN avg_balance_active IS NOT NULL
      THEN 1 ELSE 0
    END
  ) AS balance_available,

  ROUND(
    100.0 *
    SUM(
      CASE
        WHEN avg_balance_active IS NOT NULL
        THEN 1 ELSE 0
      END
    ) / COUNT(*),
    2
  ) AS balance_coverage_pct,

  /* Past due */

  SUM(
    CASE
      WHEN avg_past_due_active IS NOT NULL
      THEN 1 ELSE 0
    END
  ) AS past_due_available,

  ROUND(
    100.0 *
    SUM(
      CASE
        WHEN avg_past_due_active IS NOT NULL
        THEN 1 ELSE 0
      END
    ) / COUNT(*),
    2
  ) AS past_due_coverage_pct,
```

/* Delinquency share */

```
SUM(  
  CASE  
    WHEN delinquent_account_share IS NOT NULL  
    THEN 1 ELSE 0  
  END  
) AS delinquency_share_available,
```

```
ROUND(  
  100.0 *  
  SUM(  
    CASE  
      WHEN delinquent_account_share IS NOT NULL  
      THEN 1 ELSE 0  
    END  
  ) / COUNT(*),  
  2  
) AS delinquency_share_coverage_pct,
```

/* Utilization */

```
SUM(  
  CASE  
    WHEN avg_utilization IS NOT NULL  
    THEN 1 ELSE 0  
  END  
) AS utilization_available,
```

```
ROUND(  
  100.0 *  
  SUM(  
    CASE  
      WHEN avg_utilization IS NOT NULL  
      THEN 1 ELSE 0  
    END  
  ) / COUNT(*),  
  2  
) AS utilization_coverage_pct
```

```
FROM  
  bu_frd.model_party_behavior_monthly_2022_2025
```

```
GROUP BY  
  model_split,
```

label_3m

ORDER BY
model_split,
label_3m DESC;

También quiero un QA pequeño de utilization

Antes de usarla quiero saber si existen valores extremos:

```
SELECT  
  percentile_approx(  
    avg_utilization,  
    ARRAY(  
      0.01,  
      0.05,  
      0.25,  
      0.50,  
      0.75,  
      0.95,  
      0.99  
    )  
  ) AS utilization_percentiles,  
  
  MIN(avg_utilization) AS min_utilization,  
  MAX(avg_utilization) AS max_utilization
```

```
FROM  
  bu_frd.model_party_behavior_monthly_2022_2025
```

```
WHERE avg_utilization IS NOT NULL;
```

No vamos a capear nada todavía. Esto es únicamente diagnóstico.

Una vez tengas esos resultados, el **23C** será el paso donde reduciremos las **8,263,068** filas a aproximadamente **688,589 filas**, y construiremos simultáneamente:

```
PRIOR 9M  
RECENT / TRAILING 3M  
current-delta features
```

+

```
baseline SD  
Z-score challenger features
```

Ahí tendremos por primera vez la tabla sobre la que podremos comparar formalmente
Current formula vs Z-score.

23c

```
SELECT
  model_split,
  label_3m,

  COUNT(*) AS party_months,

  ROUND(AVG(reported_accounts),2)
    AS avg_reported_accounts,

  ROUND(AVG(active_accounts),2)
    AS avg_active_accounts,

  ROUND(
    100.0 *
    SUM(CASE WHEN active_accounts = 0
      THEN 1 ELSE 0 END)
    / COUNT(*),
    2
  ) AS zero_active_accounts_pct,

  ROUND(AVG(rated_active_accounts),2)
    AS avg_rated_active_accounts,

  ROUND(
    100.0 *
    SUM(
      CASE
        WHEN rated_active_accounts = 0
        THEN 1 ELSE 0
      END
    ) / COUNT(*),
    2
  ) AS zero_rated_accounts_pct,

  ROUND(AVG(utilization_eligible_accounts),2)
    AS avg_util_eligible_accounts,

  ROUND(
    100.0 *
    SUM(
      CASE
        WHEN utilization_eligible_accounts = 0
        THEN 1 ELSE 0
      END
    ) / COUNT(*),
    2
```

```

) AS zero_util_eligible_pct

FROM bu_frd.model_party_behavior_monthly_2022_2025

GROUP BY
    model_split,
    label_3m

ORDER BY
    model_split,
    label_3m DESC;

```

Un segundo QA de utilization que haría ahora

Necesito ver qué tan grande es realmente la cola extrema:

```

SELECT

    SUM(
        CASE WHEN avg_utilization > 1
        THEN 1 ELSE 0 END
    ) AS rows_above_100pct,

    ROUND(
        100.0 *
        SUM(
            CASE WHEN avg_utilization > 1
            THEN 1 ELSE 0 END
        ) / COUNT(*),
        4
    ) AS pct_above_100pct,

    SUM(
        CASE WHEN avg_utilization > 2
        THEN 1 ELSE 0 END
    ) AS rows_above_200pct,

    SUM(
        CASE WHEN avg_utilization > 5
        THEN 1 ELSE 0 END
    ) AS rows_above_500pct,

    SUM(
        CASE WHEN avg_utilization > 10
        THEN 1 ELSE 0 END
    ) AS rows_above_1000pct,

```

) AS rows_above_1000pct,

MAX(avg_utilization)
AS max_utilization

FROM bu_frd.model_party_behavior_monthly_2022_2025

WHERE avg_utilization IS NOT NULL;

Y quiero revisar ejemplos extremos:

```
SELECT
  party_id,
  observation_date,
  feature_month,
  model_split,
  label_3m,
  active_accounts,
  utilization_eligible_accounts,
  avg_utilization
```

FROM bu_frd.model_party_behavior_monthly_2022_2025

WHERE avg_utilization > 5

ORDER BY avg_utilization DESC

LIMIT 50;

No para excluirlos todavía, sino para entender si son excepciones legítimas.